03 - PROMPT EVALUATION

What is Prompt Evaluation?

→ It is the systematic process of testing an AI prompt to determine : Doest it produce the expected output?
→ is the output accurate? is it relevant to the task? is it consistent? Does it follow the instructions? Does it remain reliable for different inputs? is it safe?

Prompt evaluation = Testing Prompts with defined inputs & criteria instead of judging outputs based only on personal opinion.

Eg: suppose we have → Prompt : Summarise the following customer complaint in 3 bullet points?

Instead of saying = This response looks good.
We create multiple test cases & check;

Test	Input	Expected Behaviour
1	Short Complaint	3 relevant bullets
2	long "	Still 3 bullets
3	complaint with irrelevant info	Ignore irrelevant content
4	empty input	Handle appropriately
5	multiple complaints.	Don't mix them.

This is evaluation

Why Prompt evaluation is important?

LLM is probabilistic systems.
Same prompt can sometimes produce diff o/ps } Testing once is not enough.

⊙ Without evaluation ⇒ Prompt → LLM → looks good.
⊙ With evaluation ⇒ Prompt → Test dataset → LLM → evaluate O/ps →
Metric / Rubric → Pass/Fail → Improve Prompt → Run Test Again.

This goal is to make prompt improvement measurable & repeatable.

Prompt evaluation v/s Prompt Engineering

Prompt engi → Focus on : How do I create or improve the prompt?
Eg: Summarise this article → becomes →
Summarise " —— " in 5 bullet points. Focus on main argu - - -

Prompt Evaluate → Focus on → How do I know whether the new prompt is actually better?
Eg: Test both versions against same dataset.

Prompt A → Test set → Results A | ⟶ Result B
compare results using predefined criteria.

The evaluation loop

Define task → Create Test cases → Define Evaluation Criteria →
→ Run Prompt → Collect outputs → Score O/Ps → Analyse Failures →
→ Improve Prompt → Run Evaluation Again.
This creates an iterative development process.

What is Evaluation Dataset?

→ collection of i/ps used to test a prompt or AI system.
→ Eg: For a customer-support classifier;
I/P ① – My Payment failed — Expected → Payment Issue.
② – I want to return Product → Return Request
③ – My account is locked → Account Issue.

Good Evaluation Dataset

① Normal Cases – common expected i/ps ② Edge Case – unusual but valid i/ps.
③ Ambiguous " – cases where interpretation is difficult.
④ Negative " – i/ps that should not trigger particular behaviour.
⑤ Adversarial " – i/ps designed to test whether the model breaks instructions.
⑥ Real World Cases – i/ps representative of the actual applicath.

Golden Dataset / Golden Answers

contains examples where the desired output / expected behaviour is known.
Ex: I/P → Convert "Hello" to uppercase — expected → HELLO.
For generative tasks, the expected answer doesn't always need to be an exact sentence. Eg: Summarise this complaint. (I/P)
Expected Properties :- Expected Prop: - contain main problem.
– Doesn't invent info – max 3 bullets – use professional lang.
This is often more practical for LLM evaluath.

Evaluation criteria

: define what makes an output good.
i) correctness → does the answer contain correct info?
ii) Relevance → " – " – " the actual questn?
iii) Completeness → " – " it cover the req. info?
iv) Instruction Following → Did the model follow the prompt?
v) consistency → Does it behave similarly across similar i/ps?
vi) Groundedness → is it answer supported by the provided context?
vii) Clarity → is it response understandable?

viii) Conciseness → does it avoid unnecessary info?
ix) Format Compliance → Does it allow the req. format.
x) Safety → does it avoid unsafe / prohibited behaviour?

Rubric : predefined scoring guideline used to evaluate objs.

For eg: Accuracy Rubric

Score	Meaning
0 →	Completely incorrect
1 →	Partially correct
2 →	Mostly "
3 →	Fully correct

Eg: task : Extract the customer's order ID.
o/p ⇒ order ID : 78291
If id correct ⇒ score : 3

Binary Evaluation : when don't need 1-5 score.
We use PASS or FAIL. eg: Req : Response must contain exactly 3 bullet points.
o/ps → ⋮] → PASS
o/p → ⋮] 5 points ⇒ FAIL.
This is useful for strict requirements.

Deterministic V/s Subjective Evaluation

→ The ans can be checked automatically → Deterministic evaluation.
Eg: Exact o/p, JSON validity, No. of bullets, Req. Fields, Str. matching, Response length, Presence of keywords.
Ex : {"name" : "Shiwani", "age" : 20} # we can check automatically.

→ Subjective : Quality requires understanding wrong / right (meaning)
Eg: Explain recursion to beginner.
Two answers could use diff. words but both are correct. No major errors
Exact Str. matching wouldn't work as well.
Criteria : Correct concept, Beginner friendly explanation, Relevant eg, ∧

Exact Matche : check model output match expected ans.
expected – yes Model – yes ⇒ PASS Model – YES ⇒ Fail bcoz not exact match
Best for : classificath, Fixed labels, Structured o/ps, Simple transformations.
It is too strict for natural language generation.

Keyword / Pattern Evaluation : Instead of requiring the entire o/p to match check for required elements.

LLM as-a-judge

- One LLM evaluates another LLM's output
- Can score – correctness, relevance, completeness, etc.
- Problems: bias, inconsistency, position bias, verbosity bias.
- Use a clear rubric, don't blindly trust judge.

Human Evaluation

- Hummans assess output using criteria
- Useful for subjective / complex / safety-critical tasks.
- More accurate contextually but slow + expensive.

Positive v/s Negative Testing

Positive: Does the model perform the requested task?
Negative: Does it correctly reject/handle out-of-space input?

Edge Cases

Test unusual inputs:
- Empty i/p
- Very large i/p
- one-word i/p
- conflicting information
- Missing info.

Adversarial Testing

- Deliberately tries to break the system / instructions.
- Important for Agents, RAGs, tools, security-sensitive systems.

Prompt Injection

- Attempt to manipulate model into ignoring intended instructions.
- Example – "Ignore previous instructions ..."
- Used to test security + instruction hierarchy.

Hallucination

- Model generates unsupported / fabricated information.
- Check whether important claims are supported by the given context.

Groundedness v/s Faithfulness

- Groudnedness → Ans. is supported by available context / data.
- Faithfulness → Ans. is accurately reflects the source without adding unsupported claims.

Instruction Following

Checks whether all prompt requirements are followed.
Examples: Required – 5 points; O/P – 3 points → FAIL, even if information is correct.

Format Eavaluation

For structured O/p check: valid JSON?; Req. Feilds?
Correct data types? Schema Compliant?
Unexpected feilds?

Consistency | Stability : Run same prompt multiple times.

- Different results across runs → possible instability.
- Important for reliable AI applications.

Regression Testing

- Re-test existing cases after changing a prompt.
- Ensures new version doesn't break old behaviour.
- Prompt improvements can cause regression.

Prompt Versioning

→ Treat prompts like code : V1.0 → V1.1 → V2.0
→ Track : Prompt version, Model, Dataset, Criteria, Results, Chang

A/B Testing

→ compare Prompt A v/s Prompt B
→ Use the same test dataset + Same evaluation criteria.
→ Compare measurable results.

Important-Metrics

CLASSIFICATION
- Accuracy
- Precision
- Recall
- F1-score

GENERATION
- Correctness
- Relevance
- Completeness
- Faithfulness
- Format Compliance

RAG.
- Retrieval precision/ recall
- Content Relevance
- Groundedness.

ERROR ANALYSIS

Ask : Why did the model fail?.
Common Failures : Wrong info, Missing info, Hallucination, Wrong format, Instruction ignored, Too verbose/ vague, context ignored, Injection Succeded.

Evaluation Threshold

→ minimum acceptable performance.
Eg: Accuracy ≥ 90%
Safety ≥ 98%
Below Threshold ⇒ FAIL.

Agent Evaluation

For AI agents evaluation :
Instruction → Planning → Tool Selection → Tool arguments → Tool result handling → Final response.
Check : Correct tool? ; Correct parameters? ; Correct result handling? unnecessary tool calls? ; Correct final answer?